

Desenvolvimento para Dispositivos Móveis

Formulários e ListView

Universidade Federal do Rio Grande do Norte

Formulários

- Aplicativos frequentemente **exigem que os usuários insiram informações em um campo de texto.**
 - Exemplo: login com uma combinação de endereço de e-mail e senha.
- Para **facilitar o uso dos aplicativos** é importante verificar se as informações fornecidas pelo usuário são **válidas.**
 - Se o usuário preencheu corretamente o formulário, as informações devem ser processadas.
 - Se o usuário enviar informações incorretas, será exibida uma mensagem de erro amigável, informando o que deu errado.

Formulários

- Neste exemplo, vamos adicionar validação a um formulário que possui um único campo de texto seguindo os passos:
 - Criar um Formulário com uma GlobalKey.
 - Adicione um TextFormField com lógica de validação.
 - Criar um botão para validar e enviar o formulário.

Criação do formulário

- O widget Form atua como um contêiner para agrupar e validar vários campos de formulário.
- Ao criar o formulário, forneça uma GlobalKey. Isso atribui um identificador único ao seu Form. Também permite que você valide o formulário mais tarde.
- Crie o formulário como um StatefulWidget. Isso permite que você crie uma GlobalKey<FormState>() única uma vez. Em seguida, você pode armazená-la como uma variável e acessá-la em diferentes pontos.
- Fazer isso como um StatelessWidget, implicaria em armazenar essa chave em algum lugar. Como é um recurso caro, não é interessante gerar uma nova GlobalKey cada vez que o método build for executado.

Criação do Formulário

```
class MyCustomForm extends StatefulWidget {  
  const MyCustomForm({super.key});  
  
  @override  
  MyCustomFormState createState() {  
    return MyCustomFormState();  
  }  
}  
  
class MyCustomFormState extends State<MyCustomForm> {  
  
  final _formKey = GlobalKey<FormState>();  
  @override  
  Widget build(BuildContext context) {  
    return Form(  
      key: _formKey,  
      child: const Column(  
        children: <Widget>[  
          // Add TextFormFields and ElevatedButton here.  
        ],  
      ),  
    );  
  }  
}
```

Campos do Formulário

- Embora o Formulário esteja configurado, ele ainda não possui uma maneira para os usuários inserirem texto.
- Esse é o papel do widget TextFormField.
 - O TextFormField renderiza um campo de texto com design material e pode exibir mensagens de erro de validação quando ocorrerem.
- Valide a entrada **fornecendo uma função validator()** para o TextFormField.
- Se a entrada do usuário não for válida, a **função validator retorna uma String contendo uma mensagem de erro.**
- Se não houver erros, **a função validator deve retornar null.**

Campos do Formulário

```
class MyCustomFormState extends State<MyCustomForm> {  
  
  final _formKey = GlobalKey<FormState>();  
  @override  
  Widget build(BuildContext context) {  
    return Form(  
      key: _formKey,  
      child: Column(  
        children: <Widget>[  
          TextFormField(  
            // The validator receives the text that the user has entered.  
            validator: (value) {  
              if (value == null || value.isEmpty) {  
                return 'Please enter some text';  
              }  
              return null;  
            },  
          ),  
        ],  
      ),  
    );  
  }  
}
```

Campos do Formulário

- Para validar o formulário, use a `_formKey` criada anteriormente.
- É possível usar o acessor `_formKey.currentState` para acessar a `FormState`, que é criada automaticamente pelo Flutter ao construir um `Form`.
- A classe `FormState` contém o método `validate()`.
- Quando o método `validate()` é chamado, ele executa a função `validator()` para cada campo de texto no formulário.
- Se tudo estiver correto, o método `validate()` retorna **true**.
- Se algum campo de texto contiver erros, o método `validate()` **reconstrói o formulário para exibir as mensagens** de erro e retorna **false**.

Botão para Validação

- Agora o formulário está pronto, será adicionado um botão que o usuário possa tocar para enviar as informações.
- Quando o usuário tentar enviar o formulário, é necessário verifique se o formulário é válido.
- Se for, exiba uma mensagem de sucesso. Se não for (se o campo de texto estiver vazio), exiba a mensagem de erro.

Botão para Validação

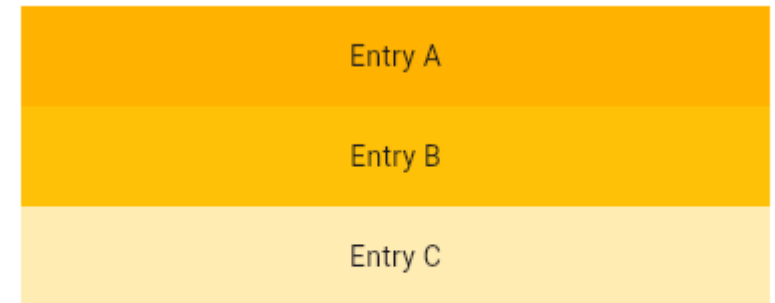
```
child: ElevatedButton(  
  onPressed: () {  
  
    if (_formKey.currentState!.validate()) {  
      ScaffoldMessenger.of(context).showSnackBar(  
        const SnackBar(content: Text('Processing Data')),  
      );  
    }  
  },  
),
```

ListView

- O ListView é o widget de rolagem mais comumente utilizado.
- Ele exibe seus filhos um após o outro na direção de rolagem.

ListView

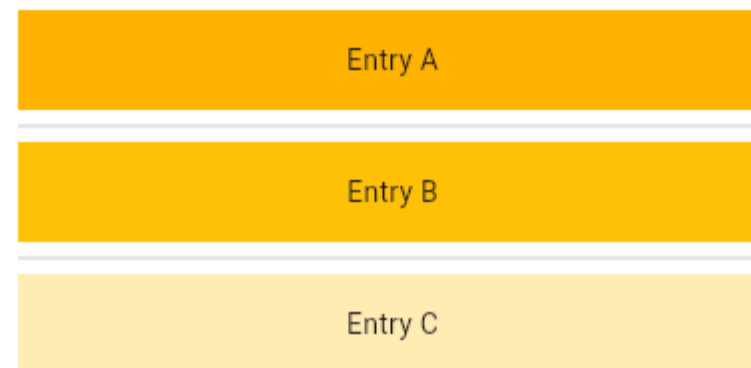
```
ListView(  
  padding: const EdgeInsets.all(8),  
  children: <Widget>[  
    Container(  
      height: 50,  
      color: Colors.amber[600],  
      child: const Center(child: Text('Entry A')),  
    ),  
    Container(  
      height: 50,  
      color: Colors.amber[500],  
      child: const Center(child: Text('Entry B')),  
    ),  
    Container(  
      height: 50,  
      color: Colors.amber[100],  
      child: const Center(child: Text('Entry C')),  
    ),  
  ],  
)
```



ListView

```
final List<String> entries = <String>['A', 'B', 'C'];  
final List<int> colorCodes = <int>[600, 500, 100];
```

```
Widget build(BuildContext context) {  
  return ListView.builder(  
    padding: const EdgeInsets.all(8),  
    itemCount: entries.length,  
    itemBuilder: (BuildContext context, int index) {  
      return Container(  
        height: 50,  
        color: Colors.amber[colorCodes[index]],  
        child: Center(child: Text('Entry ${entries[index]}')),  
      );  
    }  
  );  
}
```



ListView

```
class _ListBodyState extends State<ListBody> {
  final livroHelper = LivroHelper();
  late Future<List> livros;

  @override
  void initState() {
    // TODO: implement initState
    super.initState();
    livros = livroHelper.getAll();
  }

  @override
  Widget build(BuildContext context) {
    return FutureBuilder(
      future: livros,
      builder: (context, snapshot) {
        return snapshot.hasData ? ListView.builder(
          padding: const EdgeInsets.all(10.0),
          itemCount: snapshot.data!.length,
          itemBuilder: (context, i) {
            return ListItem(livro: snapshot.data![i]);
          },
        )
        : const Center(
          child: CircularProgressIndicator(),
        );
      },
    );
  }
}
```

```
class ListItem extends StatelessWidget {
  final Livro livro;

  const ListItem({super.key, required this.livro});

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: () {
        ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
          content: Text("Single Tap"),
        ));
      },
      onLongPress: () {
        ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
          content: Text("Long Press"),
        ));
      },
      child: ListTile(
        title: Text(livro.titulo),
      ),
    );
  }
}
```

Bibliografia

- <https://docs.flutter.dev/cookbook/persistence/sqlite>
- <https://docs.flutter.dev/cookbook/forms/validation>